EECS 348 Group 1

C++ Mathematical Compiler
Software Development Plan
Version 1.0

Revision History

Date	Version	Description	Author
10/Feb/26	1.0	Started first half of Management Process	Lauren
18/Feb/26	1.1	Finished first half of Management Process	Lauren
20/Feb/26	1.2	Finished Project Organizaiton	Hudson
21/Feb/26	1.3	Finished Introduction	Logan
22/Feb/26	1.4	Completed Sections 4.3, 4.6-4.8, 5	Collin
22/Feb/26	1.5	Completed Section 2	Quinn Sells
22/Feb/26	1.6	Cleaned up formatting & Fixed grammatical errors	Quinn Sells
22/Feb/26	1.7	Reviewed the section and prepped for deliverable	Jacob Harmsen

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Scope 4

1.3 Definitions, Acronyms, and Abbreviations 4

1.4 References 4

1.5 Overview 4

2. Project Overview 5

2.1 Project Purpose, Scope, and Objectives 5

2.2 Assumptions and Constraints 5

2.3 Project Deliverables 5

2.4 Evolution of the Software Development Plan 6

3. Project Organization 6

3.1 Organizational Structure 7

3.2 External Interfaces 7

3.3 Roles and Responsibilities 7

4. Management Process 8

4.1 Project Estimates 8

4.2 Project Plan 8

4.3 Project Monitoring and Control 9

4.4 Requirements Management 9

4.5 Quality Control 9

4.6 Reporting and Measurement 10

4.7 Risk Management 10

4.8 Configuration Management 10

5. Annexes 10

Software Development Plan

1. Introduction

1.1 Purpose

The purpose of the *Software Development Plan* is to organize the groups roles and responsibilities among the group. It's also meant to layout our deadlines for the rest of the deliverable plans and any requirements or constraints that may appear throughout the project. The project plan can also make a clear timeline and outline for any design, coding, and implementation sections.

The following people use the *Software Development Plan*:

- The **Project Manager** uses it to plan any times to meet for a specific deliverable, and remind people of deadline times
- **Project team members** can look back on the plan to find specifics on their roles or something that they would handle
- **Deadlines** can also be set on the plan and looked back on

1.2 Scope

The scope of this project is to write a compiler in C++ that can take in an equation and spit out the answer for the equation. An example would be taking in 2 + 2 or (2*2) and being able to understand all symbols and perform arithmetic in the correct order of operations (Parenthesis, Exponents, Multiplication/Division, Addition/Subtraction)

1.3 Definitions, Acronyms, and Abbreviations

- **C++ (C plus plus) -** Coding language used for the project which is an extension of the C language.
- **Compiler -** Which will translate the code into a lower level which can be executed.
- **UML (Unified Modeling Language) -** Industry standard for easy-to-read software modeling.
- **Roles -** Specific tasks and responsibilities among the group to keep the project on track, different names like project manager and reviewer are examples
- **EECS Cycle server -** A central server running Linux belonging to The University of Kansas School of Engineering EECS department where code can be executed.
- **GitHub -** Online website that allows for collaboration and multiple people to upload files whether its code or a pdf
- **Software requirements -** General guide or needs for the project that need to be fulfilled to get a grade

1.4 References

We haven't referenced any extra documents yet since most of this information is just from us dividing work and assigning roles.

1.5 Overview

This *Software Development Plan* contains the following information:

1.5.1 Project Overview

- The overview is a general map or process that can be put to paper. It lists all deliverables that can be created and mentions all the constraints.

1.5.2 Project Organization

- This section is entirely focused on the split of total work and responsibilities among the group. It goes over how work on projects will be split and everyone's different roles and what each role does like the reviewer who looks over work to make sure there's no mistakes or fixes looked over.

1.5.3 Management Process

- The Management process covers the operation and timeline of the entire project. So, things like cost, deadlines, and goals in each project are in that section.

1.5.4 Applicable Plans and Guidelines

- These are any ideas or plans that can be put to paper then actually created. Like a design that's then coded.

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

The purpose of this project is to gain an understanding of how compilers work by implementing a small-scale C++ compiler. The scope of the compiler will be mathematical operations and arithmetic like addition, multiplication, division and correct order of operations.

2.2 Assumptions and Constraints

This details the resources that will be allocated to this project to give an idea of the timeline and resulting project at this scale.

2.2.1 Staff

- The project will be completed by six engineers. The team includes a project manager, designer, two implementors/integrators, a reviewer, and a tester.

2.2.2 Schedule

- The project will be completed over the course of two months and two weeks with six deliverables including this Project Plan. Each deliverable will be completed on a biweekly basis up to the Implementation, Test Cases and User Manual Deliverables, which will all be delivered around the same time.

2.2.3 Equipment

- The team will have access to Computers with Integrated Development Environments installed and available extensions. The team will also be using a Linux compiling environment on the KU School of Engineering EECS Cycle Server.

2.3 Project Deliverables

This is a comprehensive list of each “milestone” of the project timeline that will be submitted for review to ensure continuous feedback throughout the project.

2.3.1 Software Requirements Specification

- This deliverable will be the detailed report on the specific requirements for the project to be implemented. This will include detailed functional requirements, non-functional requirements, and constraints
- Will be delivered on or before March 15th, 2026.

2.3.2 Software Architecture

- The deliverable on the architecture of the software will be the full, modeled design of the software system. It will include a UML diagram which visualizes every use case of the software and how each part will interact with one another. It will serve as the blueprint for the construction of the program.
- Will be delivered on or before April 5th, 2026.

2.3.3 Implementation

- This will be the first edition released of the working software in its entirety. All code and documentation will be included in this deliverable. Being a first edition "Beta" version, it will most likely still have some bugs needing to be fixed, and other features may still need to be implemented. This will all be recorded and implemented during the testing phase.
- Will be delivered on or before May 7th, 2026.

2.3.4 Testing

- Testing will occur after a working system has been implemented. The program will run through various test scenarios, ensuring proper functionality and robustness. This is also when we will fix any remaining bugs and troubleshoot any failed test cases.
- Will be delivered on or before May 7th, 2026

2.3.5 User Manual

- The User Manual will be delivered at the completion of the program and once a final product is delivered. It will provide directions on how to use every aspect of the software in detail.
- Will be delivered on or before May 7th, 2026.

Deliverables for each project phase are identified in the Development Case. Deliverables are delivered towards the end of the iteration, as specified in section *4.2.4 Project Schedule*.

2.4 Evolution of the Software Development Plan

This document will be updated throughout the project to account for variables during the development process. These continued revisions will include:

- A table with a code/architecture revision history and explanations of each revision.
- A revision history of this document to show all changes to any aspect of the project.

The *Software Development Plan* will be revised prior to the start of each Iteration phase.

3. Project Organization

3.1 Organizational Structure

The organizational structure of the development team is set up to be mostly equal, with major decisions being made unanimously among the group, while each person maintains their own role and the objectives associated with it. The project is overseen by a course TA, Liangqin Ren, who serves as the primary point of contact for any issues related to the project. Below the TA, the Project Manager, Jacob Harmsen, manages deadlines for each deliverable and ensures that the objectives for each deliverable are correctly assigned. The rest of the development team is responsible for completing the deliverables according to the assignments of the Project Manager. Each member of the development team has a role assigned to them where they specialize and are primarily responsible for the tasks associated with their role; however, this does not limit other development team members from assisting with their responsibilities.

3.2 External Interfaces

3.2.1 Instructional Staff

The most primary external interface is the instructional staff who are responsible for the initial requirement definition as well as the acceptance of each milestone, and the final product. They are also responsible for milestone deadlines and any conflict resolution required.

- Internal Contact: Jacob Harmsen (Project Manager), Quinn Sells (Designer)
- External Contact: Liangqin Ren (TA)

3.2.2 GitHub

GitHub will be responsible for version control and hosting our artifacts and documentation. All of our project artifacts and documentation will be organized in one repository, allowing for easy collaboration and version control.

- Internal Contact: Hudson Rupp (Implementor), Lauren Wesely (Integrator)
- External Contact: GitHub Support

3.2.3 Cycle Servers

The KU EECS cycle servers are responsible for the execution environment and deployment of the final product. These servers act as the standardized environment that the project should run on. These servers also provide a development environment, but this may not necessarily be used.

- Internal Contact: Lauren Wesely (Integrator), Quinn Sells (Designer)
- External Contact: KU IT department

3.3 Roles and Responsibilities

Person	**Unified Process for EDUcation Role**	**Primary Responsibilities**	**Email**
Jacob Harmsen	Project Manager	Resource and Priority Management	j068h985@ku.edu
Quinn Sells	Designer	System Architecture, UI	quinnsells@ku.edu
Hudson Rupp	Implementer	Source Code, Unit Tests	hudson.rupp@ku.edu

Lauren Wesely	Integrator	Build Systems, Deployment	wesely@ku.edu
Logan Sigg	Reviewer	QA Deliverables, Code Reviews	l550s255@ku.edu
Collin Rawson	Tester	Test Design, Debugging	collinrawson@ku.edu

Anyone on the project can perform any role activities. This includes assisting other development team members with their primary responsibilities if needed.

4. Management Process

4.1 Project Estimates

The estimated cost for this project is $0. None of us are paid, and EECS computers and related software are provided. This is unlikely to ever change and will not be re-estimated.

4.1.1 Scheduled Deadlines:

Project Plan: Feb 22nd

Software Requirements Specification: Mar 15th

Software Architecture: Apr 5th

Implementation: May 7th

Test Cases: May 7th

User Manual: May 7th

This will only change when the due dates by the professor are changed.

4.2 Project Plan

4.2.1 Phase Plan

- Inception
  - Project Plan: Complete the Project Plan document
  - Software Requirements Specification: Complete the SRS document
- Elaboration
  - Software Architecture: Complete the Software Architecture document
- Construction
  - Implementation: Complete the code that satisfies the earlier deliverables
  - Test Cases: Complete the test cases document and test the code until it satisfies the requirements
- Deployment
  - User Manual: Complete the User manual document
  - Transition: Release/submission: Upload materials to Canvas

4.2.2 Iteration Objectives

Only one iteration will be made, as the software is relatively simple. The mathematical compiler will be able to handle +, -, *, /, %, **, and parenthesis operations when solving a given equation and show an appropriate error when the equation is invalid.

4.2.3 Releases

The final Gold release will be able to handle +, -, *, /, %, **, and parenthesis operations. No other release will be used as this compiler doesn't have a real audience outside of the development team and grader, so no outside feedback is needed.

4.2.4 Project Schedule

Milestone	**Date**
Project Plan Document	02/22/26
Software Requirements Specification	03/15/26
Software Architecture	04/05/26
Implementation	04/21/26
Test Cases	04/30/26
User Manual	05/07/26
Release	05/07/26

4.2.5 Project Resourcing

There is one product owner, the professor, one project manager, Jacob, and the remaining 5 people are the development team. No one is particularly confident in C++ except for Quinn and we all have similar set of other skills. No formal training is needed, and any lack of knowledge can be learned on an as-needed basis.

4.3 Project Monitoring and Control

The project will be monitored and controlled as outlined in sections 4.4-4.8.

4.4 Requirements Management

Any changes to the requirements document must be agreed upon with at least 4 group members and announced to the Discord channel and the Requirements document.

4.5 Quality Control

Everything created will be checked over by the project manager, Jacob, prior to submission. The development team's work will be done at least a day before the submission to allow time for editing and revising. For more important deliverables, more time will be given for revision, particularly for implementation and testing.

4.6 Reporting and Measurement

Meeting reports will be created after each group meeting detailing the attendees, the topics discussed and their key takeaways, and the time spent meeting. In addition, upon the completion of key milestones determined by the team, logs will be made stating which key milestone has been reached, the amount of time spent on said milestone, which group members contributed to the milestone, and what each group member contributed. Because this is a timed undertaking, time is the most important metric to track, and as such, time is the primary metric tracked in this project. In addition, because time is far too broad of a metric alone, how the time is spent will also be tracked.

4.7 Risk Management

This project has a clearly defined scope with clear requirements and deadlines, and as such, these risks are negligible. Therefore, the main risk in this project is poor time management; this has already been partially addressed through timed deadlines for different parts of the project, but within those deadlines, regular meetings of the team should be held to avoid potential procrastination. The other large risk this project contains is a poor standard of work. This risk can be handled through two methods: time management and peer review. The weekly meetings and emphasis on time management will encourage team members to avoid rushed, low quality work, and through team members reviewing each other's work, a standard of quality can be upheld.

4.8 Configuration Management

All program updates will be uploaded to a shared GitHub repository, which allows for changes between versions to be tracked, backups to be maintained, and new versions to be reviewed by the project reviewer. All project artifacts will be accessed through a shared Microsoft Word document, and finalized versions will be submitted through Canvas. They will be named and done in the order specified on Canvas (Project Plan, Software Requirements Specifications, Software Architecture Document, Test Cases, User's Manual).

5. Annexes

The project will follow the UPEDU process.